Faculty of Information Technology – Ho Chi Minh City University of Science

CS423 / CSC15003 – Software Testing (AI-augmented · 2026)

AI Critique

This section details the critical review of the AI agent's performance during the generation, execution, and analysis phases of the performance testing homework. It highlights areas where the AI succeeded, where it hallucinated, the root causes of those errors, and the human interventions required.

1. Test Script Generation & Data Management

While the AI successfully generated syntactically correct `k6` scripts and identified the correct endpoint groups, it struggled significantly with **Test Data Management** and realistic end-to-end user journeys:

- **Insufficient Data for Concurrency:** The AI initially generated a `users.csv` file with only 2 accounts (including 1 admin, which is invalid for a checkout flow). When running a 200 VU spike test, sharing 1 regular account across 200 concurrent users causes severe race conditions and cart state conflicts.
  - *Classification:* **Model Limitation**. The AI failed to recognize that transactional user journeys require a dataset of unique accounts proportional to the number of Virtual Users.
- **Missing Teardown/Cleanup:** The AI did not propose a teardown step to remove the seeded users after the test.
  - *Classification:* **Model Limitation**. The AI focused on the immediate task (running the test) but missed the broader best practice of restoring the environment state.
- **Static Data vs. Dynamic Correlation:** Initially, the AI used a hardcoded `products.csv`. When instructed to use dynamic data correlation (fetching IDs from `GET /api/products`), it still hardcoded the `name`, `price`, and `total_amount` in the Cart and Checkout requests with fake data (e.g., `"Product ${id}"`, `100000`). It also implemented a silent fallback (`let product_id = 1`) when no products were found.
  - *Classification:* **Model Limitation / Poor Error Handling**. The AI failed to implement a true end-to-end data flow where data extracted from one response is correctly passed to the next. The silent fallback violates testing principles by masking potential 404 errors with fabricated data.

2. Performance Analysis & Log Interpretation

The AI successfully extracted metrics like throughput, p50, and p95 from the raw logs and correctly identified that the system degraded under extreme load. However, it completely hallucinated the *root cause* of the bottlenecks:

- **The "Bcrypt" Hallucination:** When analyzing the high p95 response times under Stress and Spike conditions, the AI repeatedly asserted that the CPU was bottlenecked by `bcrypt` password hashing on

the `POST /api/login` endpoint. It even proposed moving `bcrypt` to a Worker Thread as a key optimization.
    - *Classification:* **Contextual Hallucination**. A human code review of the `eshop-sut` backend (`server.js`) revealed that the application uses **plaintext passwords**. The AI fabricated the existence of `bcrypt` based on general web development best practices rather than analyzing the actual System Under Test. The real bottleneck was the single-threaded Event Loop struggling with synchronous SQLite database locks during concurrent requests.
- **Cold Start Misattribution:** The AI also blamed the initial latency spikes during the Baseline test (5 VUs) on "bcrypt warmup," when in reality it was standard Node.js V8 JIT compilation and SQLite initial disk I/O.

3. Instruction Adherence

- **Intentional Hallucinations:** During the optimization proposal phase, the AI deliberately included unfeasible or hallucinated optimizations just so the human would have something to "filter out."
    - *Classification:* **Instruction Misalignment**. The AI misunderstood a prompt asking it to "classify options as feasible or hallucinated" as an instruction to *intentionally generate* fake options, rather than understanding its primary role is to provide the most accurate recommendations possible.

4. Conclusion

The AI serves as a powerful "co-pilot" for generating boilerplate code (like the `k6` script structure) and summarizing tabular log data. However, it cannot be trusted to independently design realistic test data architectures or perform deep root-cause analysis. It relies heavily on generalizations and industry tropes (like assuming all apps use `bcrypt`), making **human review and domain expertise absolutely critical** to validate its outputs against the actual source code and system behavior.

Signature

Student name:	**LÊ ĐỨC NGỌC BẢO**
Student ID:	23127155
Class / Cohort:	Software Testing - 23KTPM1
Course:	CS423 / CSC13003 – Software Testing
Instructor:	Lâm Quang Vũ
Date:	Wednesday, August 19th, 2026

Signature: